

Love Letter
Projet AP4B
Version de pré-rendu

Julie BENED
Corentin HAUTEFAYE
Loïc MORIN
Théo RIGAL

19 décembre 2025

Introduction

Dans le cadre de l'UE AP4B (*Semestre A25*), nous avons réalisé une adaptation du jeu *Love Letter* en Java. L'objectif principal de ce projet est de concevoir une application fonctionnelle, robuste, clairement structurée, tout en respectant les principes fondamentaux de la programmation orientée objet.

L'accent est ainsi mis sur la conception UML, l'organisation du code ainsi que la capacité à modéliser efficacement les entités essentielles d'un moteur de jeu simple.

Par ailleurs, cette adaptation vise à préserver l'esprit et les règles du jeu original, tout en y intégrant une dimension créative et ludique en lien avec le contexte universitaire.

Table des matières

1	Présentation	4
1.1	Règles du jeu	4
1.2	Conventions	5
1.3	Outils utilisés	5
2	Réalisation du jeu	6
2.1	Choix	6
2.2	Architecture générale	6
2.3	Organisation des paquets	7
3	Conception UML	9
3.1	Diagramme de classes	11
3.2	Diagramme de cas d'utilisation	12
3.3	Diagramme de séquence	13
4	Retour d'expérience	14
4.1	Organisation	14
4.2	Difficultés rencontrées	14
4.3	Suggestions d'amélioration	14
5	Conclusion	15

1 Présentation

1.1 Règles du jeu

Love Letter est un jeu de cartes qui se joue de 2 à 6 joueurs et dont le principe est assez simple. Vous incarnez un courtisan qui cherche à transmettre une missive d'amour à la princesse, tout en éliminant ses opposants.

Mise en place Vous disposez d'un ensemble de 21 cartes personnages mélangées, face cachée ; il s'agit de la pioche. La première carte en est defauscée et est placée **face cachée** dans la pile de jeu. Si la partie comporte exactement deux joueurs, alors il convient d'enlever trois cartes supplémentaires de la pioche ; ces dernières sont placées dans la pile, **face visible**. Ensuite, une carte est distribuée à chaque joueur ; cela constitue leur main. Le premier joueur à commencer est celui qui a remporté la dernière manche. S'il s'agit de la première manche, ou qu'il y a eu égalité, il est choisi aléatoirement.

Victoire d'une manche Une manche arrive à échéance lorsqu'une des deux conditions suivantes est remplie :

- Être le dernier joueur en partie
- La pioche est vide

Dans le second cas, une fois que le tour du joueur en cour est terminé, chacun dévoile sa main. Celui qui dispose de la plus grande carte remporte la manche. Notons que les égalités sont tolérées. Enfin à l'issue de la manche, chaque vainqueur gagne un point faveur.

Victoire d'une partie On considère qu'il y a un total de treize points faveur par partie. Ainsi, elle est gagnée s'il existe au moins un joueur dont le score est d'au moins la valeur indiquée dans le tableau ci-dessous :

Nombre de joueurs	2	3	4	5	6
Score requis	6	5	4	3	3

Il est de plus à noter qu'il est toujours possible de gagner une partie avec le nombre de pions faveur de départ, en respectant ce tableau.

Tour des joueurs Le jeu s'effectue dans le sens horaire. Chaque joueur procède en trois étapes :

1. Piocher une carte du paquet
2. Choisir une de ses deux cartes
3. Jouer la carte, résoudre son effet et la place **face visible** dans la pile

Quitter la manche Lorsque le joueur est contraint de quitter la manche, il doit défausser sa main **face visible** et passe bien sûr son tour à chaque itération.

Cartes personnages Les cartes en question ont été adaptées à un contexte UTBM et sont décrites en section 2.1.

1.2 Conventions

Afin d’assurer une meilleure lisibilité ainsi qu’une cohérence globale du projet, plusieurs conventions ont été adoptées tout au long du développement.

Tout d’abord, le code source et les commentaires sont rédigés en Anglais ; cela est en effet conforme aux standards couramment utilisés en programmation, notamment en entreprise. La typographie utilisée est la norme Java : les noms de classes utilisent la notation *PascalCase*, tandis que les noms de méthodes et attributs suivent le format *camelCase*. Les constantes sont placées dans un fichier dédié (`Const.java`) et sont notées en majuscules avec des *underscores* en tant que séparateurs. En sus de cela, les classes spéciales comme les énumérateurs et interfaces, sont respectivement préfixées par un **E** et un **I**. Enfin, les commentaires suivent le format *Doxygen* ; ainsi la signature de chaque fonction/méthode est donnée en entête du corps.

Les diagrammes UML (*cf. section 3*) respectent les conventions classiques de modélisation. Les attributs, méthodes et relations sont clairement identifiées. Notons toutefois que les méthodes triviales, comme les *accesseurs* et *mutateurs*, ne sont pas représentés. De même, les classes utilitaires standard (*ex* : `java.awt.Color`) ne sont pas indiquées. Enfin, les dépendances mineures ont été simplifiées afin de privilégier la lisibilité de l’architecture globale.

1.3 Outils utilisés

- Pour réaliser ce projet, les outils suivants ont été utilisés :
- **IntelliJ IDEA** pour l’environnement de développement
 - **Git** et **GitHub** pour le contrôle des versions
 - **L^AT_EX** pour la rédaction du rapport
 - **tikz-uml** pour dessiner les diagrammes UML en L^AT_EX

Aucune version de Java n’a été spécifiée pour ce sujet. Ainsi, nous avons choisi d’utiliser Java 21. En effet, il s’agit d’une version **LTS** (*Long Term Support*), i.e une version stable pour le développement, et qui reste relativement moderne et sécurisée. *Swing* est également utilisé pour la gestion des éléments de l’interface graphique (*UI*).

2 Réalisation du jeu

2.1 Choix

ADAPTATION UTBM ICI

2.2 Architecture générale

La classe `Main` agit comme le point d'entrée du programme et instancie la classe `LoveLetter`, tout en ouvrant un processus dédié. Celui-ci procède en trois étapes :

1. Initialisation

- Initialise la gestion des entrées, des scènes et des assets
- Crée et configure la fenêtre, ici un `JFrame`
- Initialise les écouteurs sur fenêtre
- Initialise et remplit les surfaces
- Charge la scène de départ (*l'écran titre*)

2. Boucle principale (*tant que la fenêtre n'est pas fermée*)

- Met à jour les scènes et leur contenu
- Nettoie les registres des entrées
- Rend la surface de jeu à l'écran (*uniquement si nécessaire*)
- Met le fil en pause le temps approprié pour garder une fréquence de rafraîchissement stable autour de 60 images par seconde

3. Nettoyage & Fermeture

- Nettoie les écouteurs
- Se débarrasse des scènes, surface et assets

Entrées Les entrées sont gérées par la classe `InputManager`. Celle-ci propose des méthodes et écouteurs triviaux, afin de capter les propriétés des touches du clavier, ainsi que de la souris (*e.g la position du curseur, l'appui du bouton gauche, ...*). Il est important que les registres associés soient nettoyés à chaque fin de tour de boucle.

Surfaces Ce projet supporte deux types de surfaces, à savoir `GamePanel` et `UIPanel`, qui héritent de la classe `JPanel`. La première est dessinée au fond de la fenêtre, et est utilisée afin de rendre des objets et éléments graphiques de façon dynamique à l'écran. La seconde est tracée en dernier, et contient une collection de `Form`. Elle est utilisée pour rendre des éléments graphiques Swing à l'écran, et peut être utilisée par exemple, pour faire des menus ou boîtes de dialogue.

Objets Des objets quelconques sont représentés par une classe héritant de `GameObject`. Ces derniers sont stockés et ordonnancés dans une scène donnée. Un objet est rendu sur la surface de jeu, et est trié sur sa *Z-profondeur* (*le repère orthonormal usuel dans $\mathbb{R}^3$ est utilisé, d'où plus z est grand, plus l'objet est proche de la caméra*). Un objet peut procéder à diverses opérations dans sa méthode `update`, puis se rendre à l'écran suivant

un ensemble d'instructions donné, ou bien suivant un **Sprite** pour un objet de type **GameSprintedObject**.

Sprites Un sprite est vu comme une collection de sous-images (*ici, des BufferedImage*), et affiche la bonne en utilisant la valeur de l'attribut **imageIndex** de l'objet **GameSprintedObject** associé. Ainsi, utiliser le champ **imageSpeed** permet d'animer le sprite à la vitesse donnée.

Un sprite importe un fichier image, et le divise en sous-images en utilisant la valeur **imageNumber** donnée dans le constructeur. Chaque sous-image doit se trouver côte à côte et sont indicée de 0 à **imageNumber - 1**, de gauche à droite. Ainsi, la largeur effective d'un sprite (*en pixels*) est égal au résultat de la division euclidienne de **spriteWidth** par **imageNumber**. Sa hauteur **spriteHeight** reste invariante.

Un sprite peut être rendu à l'aide de sa méthode **render**, avec une échelle donnée, un angle de rotation (*en degrés*) et origine ((0,0) *par défaut*). Les techniques de *blending* et de rendu avec teinte ne sont pas prises en charge.

Scènes Un écran donné (*e.g l'écran titre*) et l'ensemble des éléments qui le compose, est représenté par une scène, i.e un objet qui implémente les méthodes définies dans l'interface **IScene**. Elle se comporte comme un automate déterministe simple, entre autres avec les méthodes **enter**, **exit**, **update** et **render**. Une scène se charge aussi de gérer les collections de **GameObject** à l'aide d'un ordonnanceur **GameObjectManager**. Elle constitue de plus la liaison entre les entrées, et les routines dédiées (*e.g, appuyer sur [espace] $\implies$ faire avancer le joueur*). Une telle classe est régie par **SceneManager**.

Gestion des assets La classe **AssetManager** gère les assets. Les types d'assets suivants sont stockés dans des tables de hachage (*nom_fichier :ressource*) :

- Effets sonores
- Sprites (*créés à partir d'images*)
- Polices d'écriture (*format de la clef : nom_police#taille*)
- Fichiers textes (*e.g TXT ou JSON*)

Les assets dans le projets se trouvent dans le dossier **resources**. Les fichiers de sprites, polices, audio et textes prennent respectivement dans leur nom les préfixes **spr_**, **fnt_**, **snd_** et **txt_**.

2.3 Organisation des paquets

Conformément aux conventions d'appellation des paquets en Java, nous avons choisi d'utiliser le domaine de premier niveau : **fr.utbm**. Tout d'abord, cela nous a semblé naturel de procéder à ce choix. Ensuite, le fait de choisir un nom unique plutôt qu'un nom banal pour un paquet, comme **shapes**, permet de travailler proprement en évitant notamment les potentielles collisions d'espace avec des bibliothèques.

Ainsi, le code source est reparti dans les dossier/paquets suivants :

- **objects**

- `objects.cards`
- `objects.player`
- `scenes`
- `sprites`
- `system` : Contient l'ensemble des classes permettant de gérer un autre type d'objet ou de ressource
- `ui` : Définition des surfaces
- `ui.forms` : Ensemble des `Form` affichées sur la surface `UIPanel`
- `utils` : Contient du code quelconque (*e.g des constantes, des calculs mathématiques, ...*)

N.B : Pour une bonne lecture, le début du nom des paquets `fr.utbm` a été omis.

3 Conception UML

Avertissement Ce projet ne se trouve pas encore au stade final de développement. De ce fait, les modélisations présentées ci-dessous constituent surtout une vue théorique de la solution. Nous avons ainsi cherché à anticiper au mieux les besoins du moteur de jeu, et par conséquent, cette architecture est susceptible d'évoluer significativement durant la dernière phase de développement, notamment si l'on doit s'adapter à des contraintes techniques ou des cas limites non identifiés lors des phases d'analyse et de conception.

Classes L'architecture (visible en section 3.1) ne suit pas un simple modèle MVC, mais repose sur le pattern *Game Loop*, orchestré par la classe centrale **LoveLetter**.

- **Le Moteur de Jeu (LoveLetter)** : Cette classe est le cœur de l'application. Elle initialise la fenêtre et centralise les gestionnaires essentiels : **InputManager** pour les entrées clavier/souris, **AssetManager** pour le chargement des ressources (sons, polices, sprites), et **SceneManager**.
- **Gestion des Scènes (SceneManager)** : Le jeu est divisé en différents états logiques (Menu, Jeu, Pause) implémentant l'interface **IScene**. Le **SceneManager** délègue les méthodes `update()` et `render()` à la scène active, permettant une transition entre les écrans.
- **Entités de Jeu (GameObject)** : Tous les objets interactifs (Joueurs, Cartes) héritent de la classe abstraite **GameObject**.
 - La classe **Player** encapsule la main du joueur et son score.
 - La classe **Card** hérite de **GameSpritableObject** (pour la gestion des sprites) et définit la méthode abstraite `playEffect()`, assurant le polymorphisme des effets de cartes.
- **Interface Utilisateur (UIPanel)** : L'interface graphique superposée (HUD, boutons) est gérée distinctement du rendu du jeu.

Utilisation L'action principale « **Jouer son tour** » implique séquentiellement :

1. **Piocher une carte** : Via la méthode `Player.drawCard()`.
2. **Consulter sa main** : Le joueur visualise ses options.
3. **Choisir une carte** : Via `Player.chooseCard()`, qui déclenchera ensuite l'effet associé.

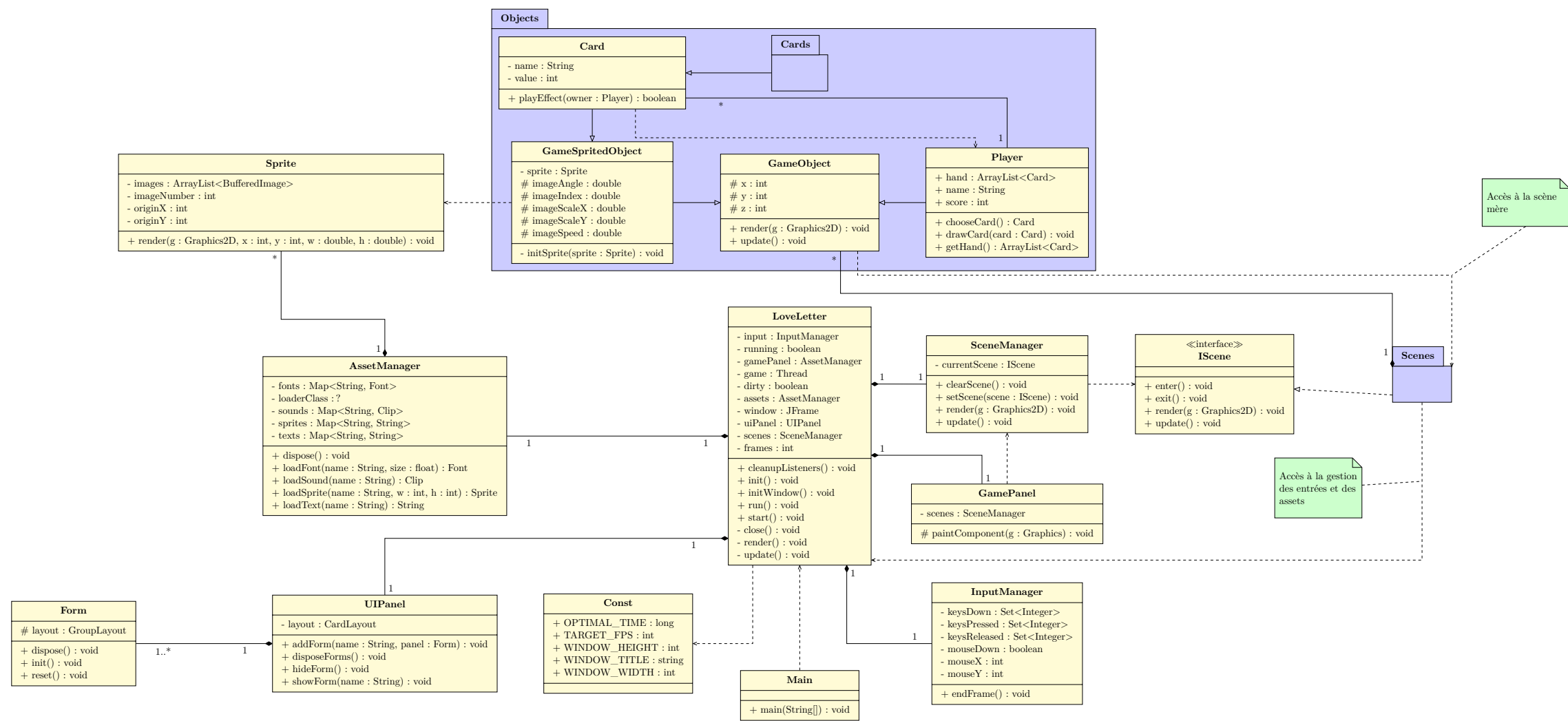
Par ailleurs, la navigation dans les menus est découplée de la logique de partie, permettant de démarrer ou quitter le jeu proprement.

Séquence

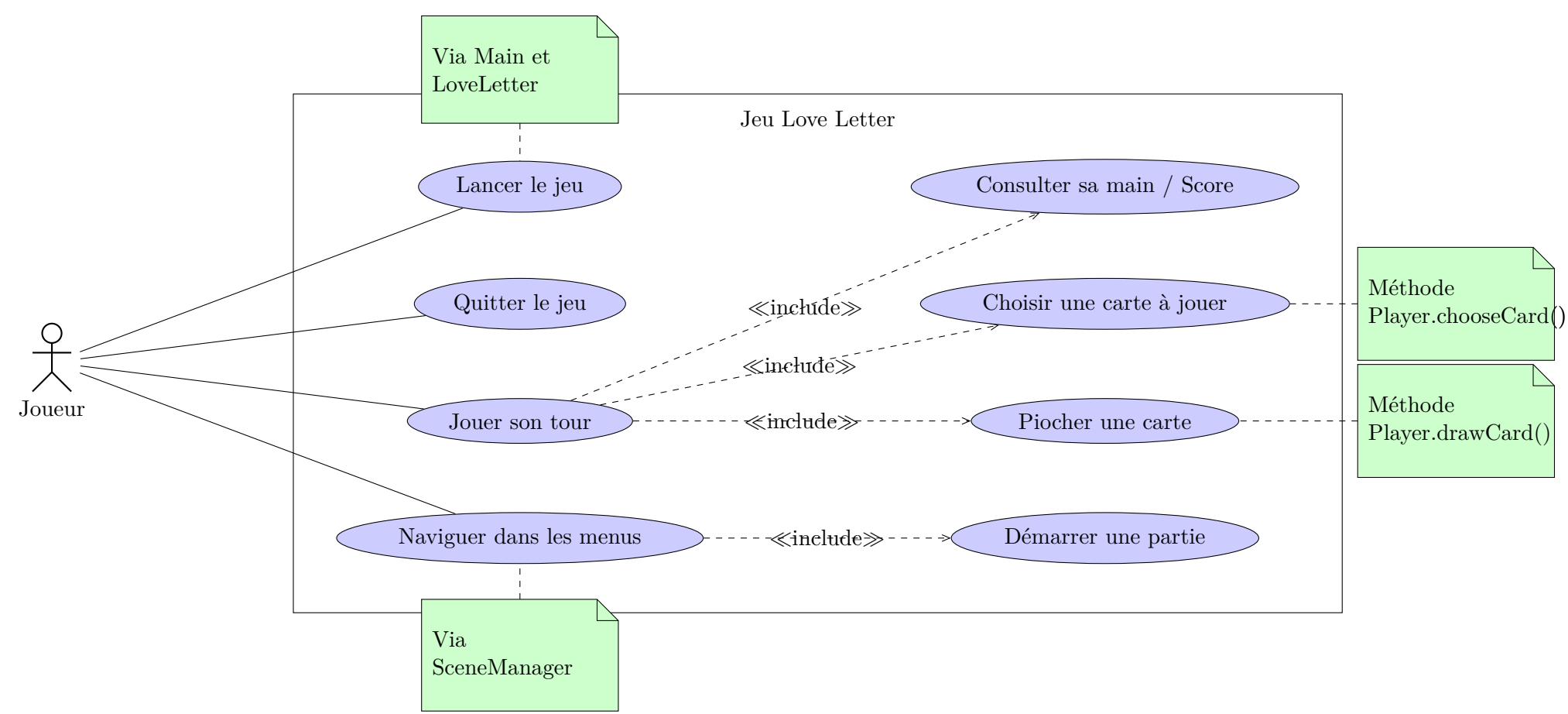
1. **Cycle de mise à jour (Update)** : À chaque *frame*, la classe principale **LoveLetter** demande d'abord au gestionnaire d'entrées (**InputManager**) de capturer l'état des périphériques (souris, clavier).
2. **Propagation** : L'ordre de mise à jour est ensuite transmis au **SceneManager**, qui le délègue à la scène active (ici, la phase de Jeu).

3. **Interrogation** : La scène interroge activement l'**InputManager** pour savoir si une action a eu lieu (ex : **isMouseDown**).
4. **Evenement** : Si un clic est détecté sur une carte, la scène sollicite l'objet **Player** pour valider la sélection via **chooseCard()**, avant d'appliquer l'effet de la carte.

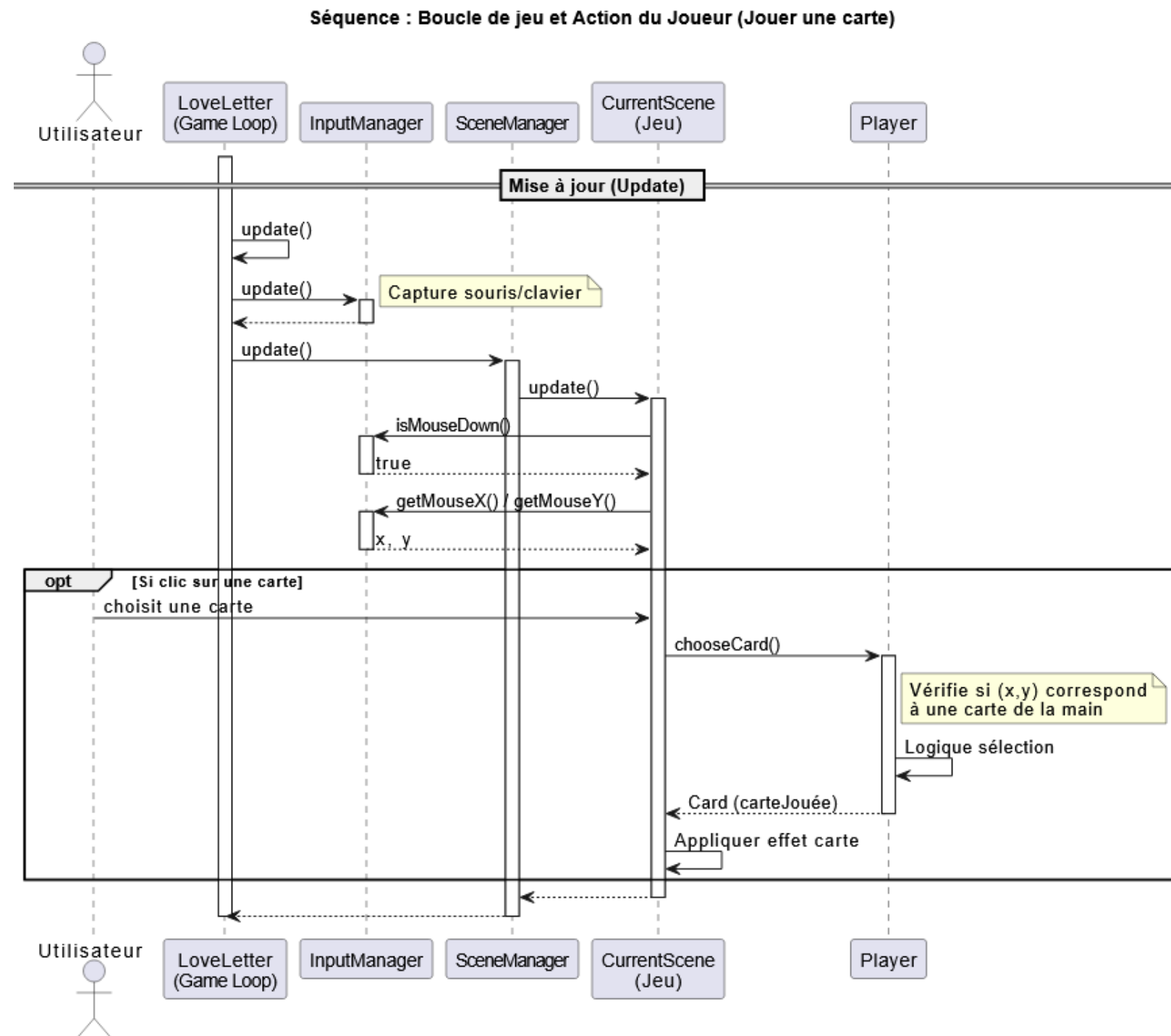
3.1 Diagramme de classes



3.2 Diagramme de cas d'utilisation



3.3 Diagramme de séquence



4 Retour d'expérience

Avertissement Cette prochaine section ainsi que la conclusion ne sont pas terminées. En effet, il serait fort mal venu de faire un retour d'expérience et/ou une conclusion sur un projet qui n'est encore pas finalisé, le but étant de mener une analyse réflexive sur le travail qui a été effectué. Nous vous remercions pour votre compréhension.

4.1 Organisation

Outils & Versionnage Nous avons utilisé **GitHub** afin de centraliser le code tout en assurant un suivi régulier du projet. Cela nous a ainsi permis de travailler sur une base commune une fois l'implémentation commencée.

De plus, nous avons cherché à nous placer dans le contexte du travail collaboratif en entreprise, et de ce fait, nous avons travaillé par *Pull Requests*. La branche **master** a été ainsi protégée par défaut des téléversements non approuvés au préalable. Aussi, nous avons procédé de la manière suivante pour la gestion des branches :

- **master** : le code source qui contient une version stable du code
- **dev** : la branche d'expérimentation où sont développées les fonctionnalités en cours de conception
- **nom-de-fonctionnalite** : branche issue de **dev**, où une fonction clef est conçue. Elle est ensuite fusionnée avec **dev**.
- **docs** : branche où la documentation est mise à jour. Le code $\text{\LaTeX}$ de ce rapport s'y trouve. Elle est repliée ensuite sur **master**.

Conception collaborative synchrone Pour la modélisation UML, nous avons préféré effectuer des réunions en présentiel plutôt que de travailler uniquement à base de commits via GitHub.

Entre les réunions, des phases de réflexion individuelles ont été menées afin d'analyser le sujet en profondeur et d'identifier les zones moins évidentes du sujet. Ces éléments ont ensuite été traités lors de ces entrevues, ce qui nous a permis de réaliser des diagrammes tout en tenant compte d'un maximum d'informations sur les points les plus complexes du sujet.

Ainsi, nous avons pu confronter les visions de tous les membres de l'équipe, puis commencer l'implémentation du jeu sur une architecture propre atteinte par consensus.

4.2 Difficultés rencontrées

[SECTION NON TERMINEE]

4.3 Suggestions d'amélioration

[SECTION NON TERMINEE]

5 Conclusion

[SECTION NON TERMINEE]